

İZMİR UNIVERSITY OF ECONOMICS

2025-2026 SPRING

SE 226 PROJECT DESCRIPTION

In this project, a Python desktop application (PDA-226) will be built that integrates the Tkinter GUI library, REST API consumption, large language models, and AI-based image generation. The application transforms a user's free-form text (a journal entry, a mood, a paragraph) into a fully imagined fictional album: cover artwork, album name, artist name, and a curated tracklist made of REAL songs that match the vibe.

In PDA-226, the user writes a short journal entry or mood description into a text area. After choosing a few customization parameters (genre, era, track count), the application sends the text to the Gemini LLM to generate album metadata and Last.fm-compatible tags. These tags are then used to query the Last.fm API for real songs matching the mood. In parallel, a generated visual prompt is sent to an AI image generation service (Pollinations.ai by default) to produce album cover art. Finally, all results are presented in a Spotify-style mockup inside the Tkinter window.

Application Parameters

To customize the album generation, the user interface provides a list of parameters and enables the user to make a choice for each (default values are used if not chosen). The parameters that influence the LLM prompt and the final output are:

Album Generation Parameters (AGP):

- **AGP1)** Journal text / mood description (free-form text area, multi-line)
- **AGP2)** Music genre (dropdown: Pop, Rock, Hip-Hop / Rap, Electronic, Indie, R&B / Soul, Jazz, Metal, Türk Pop, Klasik)
- **AGP3)** Era (dropdown: 1970s, 1980s, 1990s, 2000s, 2010s, 2020s)
- **AGP4)** Track count (spinbox: 6 to 14)

These parameters are forwarded to Gemini, which returns a structured JSON object containing the album name, artist name, year, label, mood description, a cover image prompt, and a list of Last.fm tags. The Last.fm tags are then used to query real tracks from Last.fm. The cover image prompt is forwarded to the image generation service to produce the artwork.

Technical Details and Requirements

REQUIREMENT 1. To fetch real tracks based on the mood, the requests library must be used to query the Last.fm Web API (tag.gettoptracks endpoint). The application must call this endpoint for the tags suggested by Gemini and combine results across multiple tags to build a varied tracklist.

REQUIREMENT 2. Tkinter must be used to build the GUI. The use of ttk widgets (ttk.Combobox, ttk.Spinbox, ttk.Button) is recommended for a modern look. Custom themes (e.g. using ttk.Style with the 'clam' theme) are encouraged but not required.

REQUIREMENT 3. The application must allow the user to enter a free-form journal entry in a multi-line text area, and to select a music genre, era, and track count from input widgets. Each parameter must have a sensible default value so that the application works even if the user does not change them.

REQUIREMENT 4. Pressing the "Generate Album" button must forward the journal text and parameters to Gemini, which must return album metadata as a JSON object (album name, artist name, year, label, mood description, cover prompt, list of Last.fm tags). The application must parse this JSON and handle parsing errors gracefully (Gemini sometimes wraps JSON in markdown code fences — these must be stripped).

REQUIREMENT 5. Using the Last.fm tags returned by Gemini, the application must query the Last.fm API to retrieve real songs. Duplicate tracks must be filtered out. The final tracklist must contain the user-requested number of tracks.

REQUIREMENT 6. The cover prompt returned by Gemini must be combined with the chosen genre's visual style description and forwarded to an AI image generation service (default: Pollinations.ai; alternatives are listed below). The returned image must be displayed in the GUI.

REQUIREMENT 7. The application must present the generated cover art, album metadata, and tracklist in a Spotify-style layout. Each track row must display the song title and artist name. Each track must include a clickable "Listen" button that opens the song's Last.fm page in the user's web browser.

REQUIREMENT 8. The application must offer a save feature that exports the album as both a JSON file (containing all metadata + tracklist) and a PNG file (the cover image) into a folder chosen by the user.

REQUIREMENT 9. Long-running operations (Gemini API call, Last.fm queries, image download) must run in a background thread (threading.Thread) so that the GUI does not freeze. A status label must inform the user about the current step ("Gemini is thinking...", "Fetching tracks...", "Generating cover...").

Setup Notes for Students

Free Gemini API key:

Visit <https://aistudio.google.com/apikey>, sign in with a Google account, and click "Create API key". No credit card or billing setup is required. The free tier is more than enough for this project.

Free Last.fm API key:

Visit <https://www.last.fm/api/account/create>, fill in the form (any reasonable values for application name and description). You will receive an API key immediately. Only the API key is needed — the shared secret is not used in this project because we only call public endpoints.

Free image generation (Pollinations.ai — default):

Pollinations.ai is a free AI image generation service that requires no API key, no signup, and no rate limit configuration. You simply construct a URL with your prompt embedded in it and perform a GET request. The response body is the image bytes. This is the simplest possible setup and is the recommended default for this project.

Alternative image generation services (if you prefer):

- **Gemini Imagen / Nano Banana** — uses the same Gemini API key, free quota of 500 images per day, generally higher quality. Slightly more code than Pollinations.
- **Hugging Face Inference API** — free tier available with a Hugging Face account, but slower and rate-limited.
- **Stable Diffusion (local)** — fully free and offline, but requires a GPU and significant setup. Not recommended for a one-semester project.

If you choose to use a service other than Pollinations, you must clearly explain your choice in the project report.

Code Samples

The following code snippets are provided to help you get started. You may use them as-is, adapt them, or replace them with your own implementation.

(1) Last.fm API request — fetch tracks by tag

Last.fm exposes a single base URL and uses the "method" query parameter to choose the endpoint. Below is the core function for fetching tracks by tag. This is the equivalent of last year's Cinemagoer code: you may use it directly without modification.

```
import requests
LASTFM_BASE_URL = "https://ws.audioscrobbler.com/2.0/"
LASTFM_API_KEY = "YOUR_LASTFM_API_KEY_HERE"

def fetch_tracks_by_tag(tag, limit=10):
    params = {
        "method": "tag.gettoptracks",
        "tag": tag,
        "limit": limit,
        "api_key": LASTFM_API_KEY,
        "format": "json",
    }
    headers = {"User-Agent": "AlbumCoverStudio/1.0"}
    response = requests.get(LASTFM_BASE_URL, params=params,
                           headers=headers, timeout=15)
    response.raise_for_status()
    data = response.json()
    return data.get("tracks", {}).get("track", [])
```

Each track in the returned list contains fields such as name, artist (a nested dict with the artist's name), and url (the Last.fm web page for the song). You can use the url field to let users open the song in their browser.

(2) Gemini API — structured JSON output

Configuring Gemini and creating a model:

```
import google.generativeai as genai
genai.configure(api_key=YOUR_API_KEY)
model = genai.GenerativeModel(model_name="gemini-2.5-flash")
```

To get structured output (JSON) from Gemini, write a clear prompt that specifies the schema and asks the model to return ONLY valid JSON. Example:

```
prompt = f"""Based on this journal entry, return ONLY valid JSON
with this schema:
{{
  "album_name": "string",
  "artist_name": "string",
  "lastfm_tags": ["array of 4-6 lowercase Last.fm tag strings"]
}}
```

```
Journal: \"{journal_text}\"
"""

response = model.generate_content(prompt)
text = response.text.strip()
# Strip markdown fences if Gemini added them
if text.startswith("```"):
    text = text.split("```")[1]
    if text.startswith("json"):
        text = text[4:]
    text = text.strip().rstrip(" ").strip()

album_data = json.loads(text)
```

(3) Pollinations.ai — image generation by URL

Pollinations is the simplest possible image generation service — no API key, no SDK, just a URL. You URL-encode your prompt and append it to the base URL. The GET response body is the image bytes.

```
import requests
from PIL import Image
import io
from urllib.parse import quote

def generate_cover(prompt):
    encoded = quote(prompt)
    url = (f"https://image.pollinations.ai/prompt/{encoded}"
           f"?width=600&height=600&nologo=true")
    response = requests.get(url, timeout=90)
    response.raise_for_status()
    return Image.open(io.BytesIO(response.content)).convert("RGB")
```

(4) Alternative — Gemini Imagen

If you prefer to use Gemini's own image model (higher quality, same API key, free 500/day quota), you can replace Pollinations with the following:

```
from google import genai

client = genai.Client(api_key=YOUR_API_KEY)
response = client.models.generate_images(
    model="imagen-3.0-generate-002",
    prompt="album cover, dreamy aesthetic, ...",
    config={"number_of_images": 1, "aspect_ratio": "1:1"}
)
# response.generated_images[0].image is a PIL-compatible image
```

Key Points to Remember

- You are expected to deliver a working program AND a project report. The report must cover the requirements, and you must declare whether each requirement is satisfied or not. If satisfied, you must provide the relevant code snippets and screenshots.
- You are free with your GUI design. You may use ttk widgets, custom colors, custom fonts, and any layout you prefer, as long as Tkinter is the underlying framework.
- If you use any material from other sources (online tutorials, Stack Overflow, GitHub), you must give references to them in your report. Each project group must work by themselves.
- Be aware that the album, artist, year, and label generated by Gemini are FICTIONAL — they are creative outputs of the LLM and do not correspond to any real release. Only the tracklist contains REAL songs (from Last.fm). This duality is part of the project's concept and should be made transparent in your UI.
- The Last.fm BASE URL is <https://ws.audioscrobbler.com/2.0/>. All Last.fm API methods are accessed by sending GET requests to this single URL with different query parameters. Define this URL once as a constant in your code.

Evaluation

Each group must choose **one group representative** for the project submission. Only the selected group representative is responsible for uploading the project report, presentation recording, source code, and any required project files. Other group members should not upload separate copies of the project. The submitted report must still include the full names and student IDs of all group members. The representative's submission will be accepted as the official submission for the entire group.

The project report must include the full names and student IDs of **all group members**. If a group member's name is not included in the submitted project report, that student will receive **zero points** for the project, even if they contributed to the work.

Criterion	Points	Description
GUI and User Experience	10p	Functional Tkinter interface, input widgets, Spotify-style album layout, clear output display, and clickable "Listen" buttons.
Gemini Integration and Album Generation	20p	Correct use of Gemini to generate album metadata, mood description, cover prompt, and Last.fm-compatible tags in valid JSON format.
Last.fm API and Real Tracklist Generation	20p	Correct use of the Last.fm API to retrieve real songs, combine results from tags, remove duplicates, and create the requested number of tracks.
Cover Image Generation and Technical Functionality	20p	Successful album cover generation, image display in the GUI, background threading, status updates, error handling, and JSON/PNG export.
Project Report and Presentation	20p	Clear report with screenshots, code snippets, list of requirements, task distribution, and a detailed explanation of what each group member contributed Important notes on Project Report: 1- For each project requirement, the report must clearly state whether the requirement is Satisfied or Not Satisfied. For every requirement, students must also provide the relevant code snippet and, if applicable, a GUI screenshot showing the result. 2- The report must include a separate section explaining the contribution of each group member. For each student, the report should clearly describe which part of the project they worked on, which files/functions they implemented or modified, and what problems they solved. The presentation should demonstrate the working application. All group members must actively participate in the project presentation recording. Each member is expected to appear in the recording and explain their own contribution to the project. The presentation recording must be no longer than 10 minutes. Any group member who does not appear in the presentation recording will not receive a project grade, even if their name is included in the report or submitted files. The presentation must clearly show that every member has contributed to and understands the submitted project.
Individual Group Member Contribution	10p	Each group member must clearly explain their own contribution, including implemented parts, worked-on files/functions, solved problems, and contribution to the final application. (both presentation and report will be used to assess individual contribution)

SAMPLE GUI

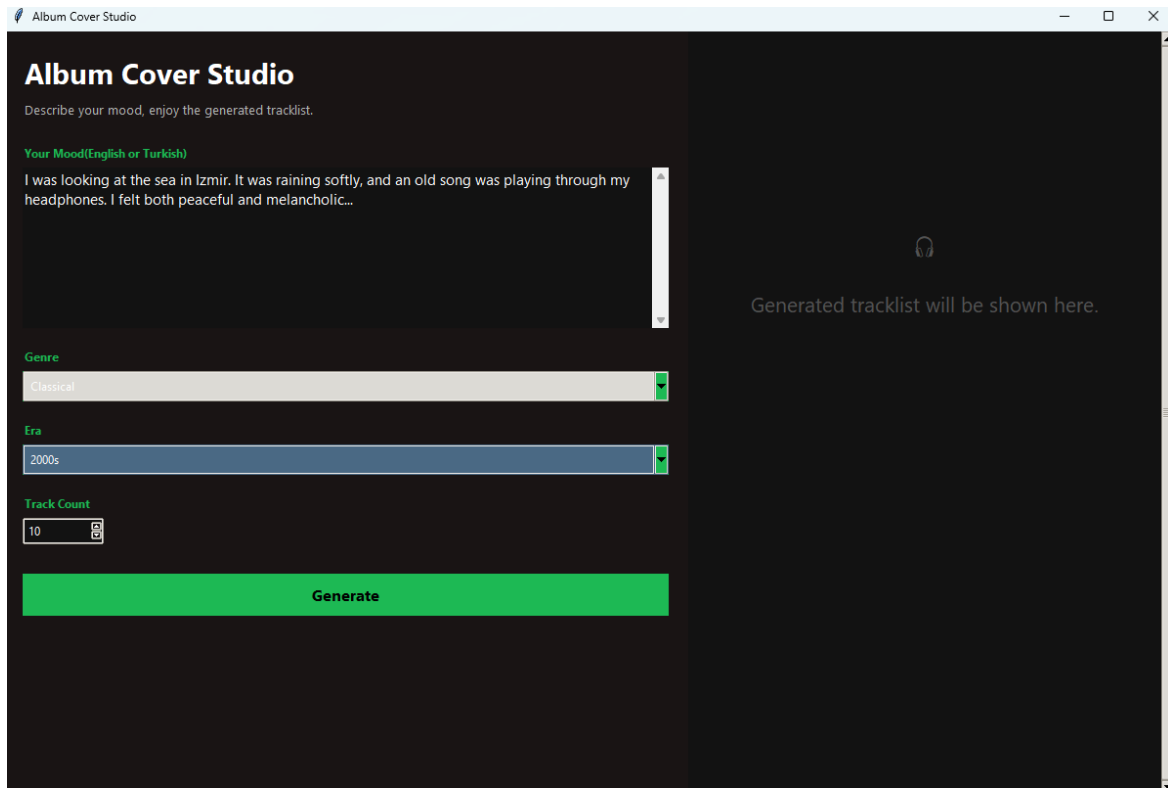


Figure 1: Initial state of application

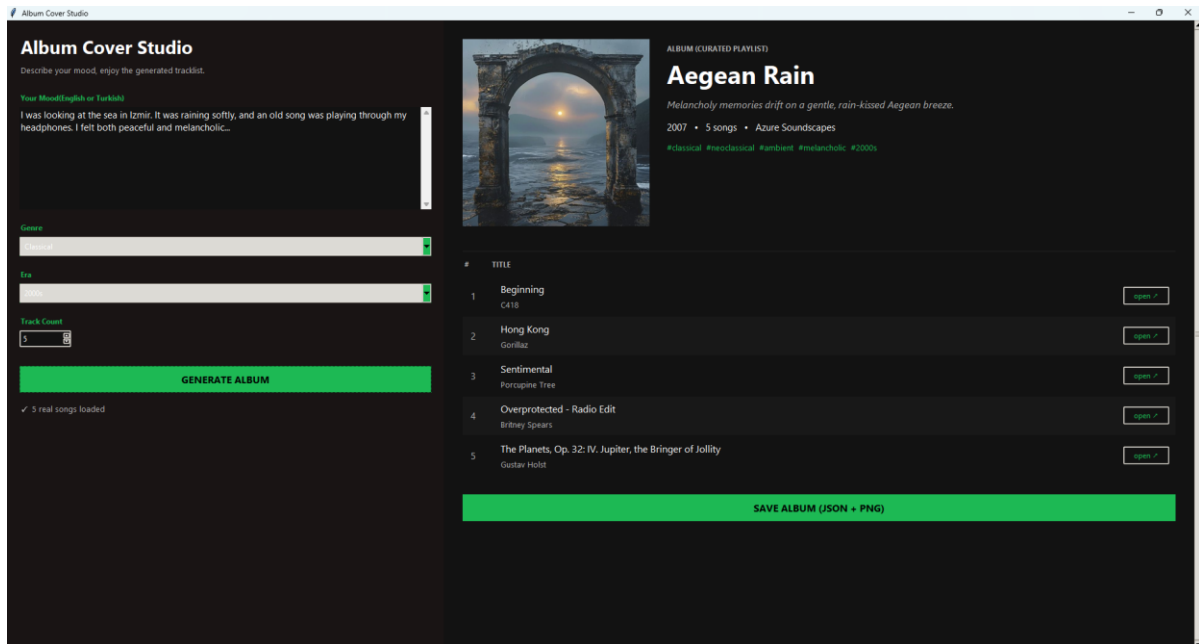


Figure 2: The application after generating an album.

IMPORTANT NOTE: You are not expected to recreate the exact same GUI shown in the example. You may design your own interface as long as the main functionality is preserved. Also, since this project uses AI-generated content and external song recommendations, the generated album cover may sometimes look unusual or meaningless, and some recommended songs may not perfectly match the selected mood or genre. This is normal and should not discourage you. The main goal is to build a working creative application that combines user input, AI-generated album concepts, image generation, and real song data.

SUBMISSION DEADLINE

The final project submission must be completed by **May 31, 2026, 23:59 Turkey time**.

Only the selected group representative should upload the project report, presentation recording, source code, and all required project files.